

Finance OS – Das Buch

Wie eine private Finanzanwendung in drei Tagen entstand – und warum sie so gebaut wurde, wie sie gebaut wurde.

Version 1.0.0 · Privates persönliches Projekt · Druckfertige Fassung (2026-07-21)

Vorwort

Dieses Buch ist die erzählende, in sich geschlossene Fassung des Finance-OS-Wissensarchivs. Es richtet sich an Leser, die das Projekt am Stück verstehen wollen – vom Anlass über die Architektur bis zur Finalabnahme. Wer stattdessen nachschlagen will, greift zur Quick Reference; wer jede Einzelentscheidung sucht, zum Decision Log; die maximale Detailtiefe je Modul bietet die Development History. Alle Aussagen dieses Buchs stammen aus den Projektdokumenten und dem Sitzungswissen der Entwicklung; nichts ist erfunden, Lücken sind als solche benannt.

Kapitel 1 – Der Anlass

Am Anfang standen zehn Markdown-Dateien und eine 94-MB-PDF: die realen Finanznotizen eines Werkstudenten – Depotpositionen bei Trade Republic, ein VL-Vertrag bei FNZ, ein Arbeitgeber-Aktienprogramm (Telekom/Shares2you über Equatex), Tagesgeld, eine zweckgebundene Rücklage. Die Notizen widersprachen sich in Details (eine Sparrate war veraltet; ein 160-€-Dauerauftrag ließ sich aus den notierten Teilen nicht zusammensetzen) und enthielten eine tückische Doppelzählungsfalle: 75 € monatliche Rücklage und 1.000 € jährliche Aktienprogramm-Einzahlung sind **dasselbe Geld** zu zwei Zeitpunkten.

Die Entscheidung, daraus eine Anwendung zu bauen, fiel mit drei nicht verhandelbaren Bedingungen: vollständig lokal (keine Cloud, keine Konten, keine Netzwerkzugriffe), fachlich ehrlich (keine erfundenen Werte, keine Doppelzählungen, keine Scheinpräzision) und ohne jede Automatik nach außen (die App empfiehlt, sie handelt nie).

Kapitel 2 – Erst verstehen, dann bauen

Die ersten fünf Phasen (alle am ersten Tag) produzierten keinen einzigen Codezeile Anwendung:

- **Quellenanalyse:** Jede Zahl aus den Notizen wurde in eine „Source Map“ überführt – mit Statusbewertung und einem Konfliktregister (K1–K16). Ein Prüfer fand dabei sogar einen Rechenfehler in den Originalnotizen (9,14 % statt korrekt 9,41 %).
- **Zwei Entscheidungsrunden mit dem Nutzer (U1/U2):** Jeder Konflikt wurde entweder verbindlich entschieden oder ausdrücklich zurückgestellt – am Ende war das Register leer.
- **Anforderungen:** 14 Module (M1–M14), explizite Nicht-Ziele und zwölf globale Regeln G1–G12, die fortan jedes Review prüfte.
- **Datenmodell und Speicherkonzept:** schemaVersion, Zuflussarten-Taxonomie, unveränderliche Snapshots, Validierungsregeln, Migrationsstrategie – plus eine Beispieldatei ausschließlich aus bestätigten Werten.
- **Der Formelkatalog F1–F22:** Jede Kennzahl mit Eingaben, Randfällen und **exakt nachgerechneten Sollwerten** (Depot 2.522,47 €; Gesamtvermögen 3.150,06 €; Telekom-Anteil 54,27775 %; Σ Kaufbedarf = Σ Verkaufbedarf = 1.573,80 €).

Diese Reihenfolge war die wichtigste Einzelentscheidung des Projekts: Der spätere Code wurde gegen fixierte Zahlen gebaut. Ein Implementierungsfehler konnte gar nicht anders, als einen Test zu brechen.

Kapitel 3 – Das Fundament (Speicher zuerst)

Phase 6 richtete das Gerüst ein – React, TypeScript, Vite, Vitest – und verzichtete bewusst auf alles Weitere: kein Router (neun feste Seiten brauchen keinen), kein UI-Kit, keine Chart-Bibliothek. Zwei Laufzeit-Abhängigkeiten, Punkt.

Phase 7 baute dann nicht etwa das Dashboard, sondern den **Speicherweg**: Öffnen per File-System-Access-API (mit ehrlichem Download-Fallback für Firefox), vierstufige Validierung mit deutschen, pfadgenauen Fehlermeldungen, Import nur mit Vorschau und Bestätigung, Export-Rundreisen, Ungespeichert-Anzeige, beforeunload-Schutz. Der Grundsatz G10 – ein fehlgeschlagener Ladevorgang verändert **nie** den Bestand – wurde hier als Code und als Testfamilie geboren.

Eine eigene Nachprüfungs-Phase (7a) verglich das Handle-Verhalten gegen fünf ausformulierte Erwartungen und fand drei echte Fehler, die die Testsuite strukturell nicht sehen konnte – darunter einen falschen Feature-Check, der das Speichern in bestimmten Browsern still verschluckt hätte. Die Lehre: Browserdialog-Verhalten braucht explizite Verhaltens-Pins.

Kapitel 4 – Die reine Finanzschicht

Phase 8 implementierte den Formelkatalog als reine, React-freie Funktionen: Vermögen (F1–F7), Sparleistung real/geglättet (F8–F10), Zielvergleich und Schwellen (F11–F16), Sparraten-Rebalancing (F17/F18), sichtbare Rundung (F19), Projektion ohne Kursentwicklung (F22). Der finance-analyst rechnete alles unabhängig nach – und fand den bis dahin größten Fehler: Die Empfehlungslogik erwähnte die Verkaufsoption auch bei **Untergewichtung**, und ein Test hatte genau dieses falsche Verhalten festgeschrieben. Beides wurde korrigiert; seither gilt die Doppellehre, dass auch Tests falsch sein können und dass Reviews Sollwerte brauchen, nicht Plausibilität.

Kapitel 5 – Vierzehn Module

Auf dem Fundament entstanden die Fachmodule – jedes durch dieselbe Prüfkette (Spezifikation → data-architect → Implementierung → Fach- und A11y-Review → calculation-tester → reviewer):

- **Konten (M8)** etablierte die Kein-Löschen-Linie (deaktivieren statt löschen – Referenzen bleiben stabil) und verdrahtete den Snapshot-Schutz.
- **Depot (M9)** brachte die Ganz-oder-gar-nichts-Snapshot-Erfassung: alle aktiven Positionen und Tagesgeldkonten zu einem Datum, danach gesperrt, Korrektur nur als neuer Snapshot. Fehlende Werte werden nie als 0 gespeichert.
- **Übersicht (M7)** übersetzte „nichts erfinden“ in die Anzeige: Ohne erfasste Werte steht dort „unbekannt“, nie eine leere Null – und die Zeit kommt seither als injizierter Stichtag in den Code, nicht aus der Systemuhr.
- **Sparrpläne (M10)** machten die Zuflussarten bedienbar und fixierten mit Nutzerentscheidung **U3** die Bezugsgröße des 1.000-€-Ziels: die eigene **reale** Monatsrate (11,85 % beim Start), geglättete Werte nur als gekennzeichnete Analyse. Eine geforderte Schätzsumme („ca. 260–265 €“) wurde bewusst **nicht** angezeigt – ohne erfasste Istwerte wäre sie erfunden gewesen.
- **Ziele (M12)** formulierten das Statusprinzip „gespeichert wird Nutzerwille, abgeleitet wird Wahrheit“ und schlossen mit **U4** die letzte Sparleistungs-Grauzone (variable Pläne zählen nur mit gültigem positivem Festbetrag).
- **Rebalancing (M11)** blieb strikt Empfehlung: drei Schwellenstufen, Konjunktiv-Pflicht („... könntest du ...“), speicherbare Planungen, die als „geplant, nicht ausgeführt“ gekennzeichnet sind und nach Verwerfen verworfen bleiben.
- **Simulator (M13)** rechnet Projektionen, keine Prognosen: nur das Depot wird verzinst, der erste projizierte Monat ist der Folgemonat des Stichtags – eine Korrektur des data-architect an der Spezifikation selbst, abgesichert durch einen Pin, der den Fehler nicht überlebt hätte.

- **Einstellungen (M14)** schlossen den Kreis: Notgroschen mit Übersteuerung (nie ungefragt ersetzt), globale Profilauswahl, und das Auto-Backup, das nach jedem Speichern exakt die geschriebenen Bytes als datierte Kopie herunterlädt – mit der Regel, dass ein Backup-Fehler niemals das Speichern scheitern lässt.

Kapitel 6 – Qualität als Prozess

Die Prüfkette war kein Ritual, sondern Fehlerfänger: Sie korrigierte zweimal die Spezifikation des Orchestrators selbst (Projektionsstart, Backup-Default), erzwang Kennzeichnungen und fand am Projektende drei Zusagen, die alle Modul-Reviews überlebt hatten, weil jedes Review nur sein Modul sah – ein versprochenes Kennzahlen-Trio, ein Tastenkürzel, eine Verlaufsansicht. Die V1-Abschlussprüfung (M15) beantwortete das nicht mit hektischem Nachbauen, sondern mit begründeten Ausweisen und einer konsolidierten Restpunktliste.

Die Zahlen dazu: 815 Tests in 37 Dateien, gewachsen in dreizehn dokumentierten Schritten von 2 auf 815, ohne dass je ein Test gelöscht wurde, um einen Fehler zu verdecken. Byte-Identität diente dabei als wiederkehrendes Orakel: Ein No-Op muss ein identisches Serialisat erzeugen, ein Backup exakt die geschriebenen Bytes enthalten.

Kapitel 7 – Release und Abschied

M16 gab dem Projekt Windows-Werkzeuge (Start/Stop mit PID-Sicherheit, Prüf-, Backup- und Release-Skripte mit Manifest aus ausschließlich real gemessenen Werten) – und die wichtigste späte Einsicht: Ein „Quellcode-Backup“ dieses Projekts ist **selbst vertraulich**, weil Dokumentation, Tests und Beispieldatei auf den realen Finanzwerten beruhen. Die Zusicherungen wurden der Realität angepasst, nicht umgekehrt.

M17 schloss ab: Secrets-Scans über Arbeitskopie, gesamte Git-Historie und die PDF (per Stream-Extraktion) fanden keine Geheimnisse; vier Finalreviews (Datenarchitektur, Security/Privacy, Barrierefreiheit, Gesamtreview) gaben ihr Ja; ein finaler Commit erhielt den annotierten, ausschließlich lokalen Tag v1.0.0. Nichts wurde gepusht, nichts veröffentlicht. Der Schlusssatz der Abnahme steht wörtlich im Protokoll: *Finance OS v1.0.0 ist lokal final freigegeben. Das Projekt wurde nicht veröffentlicht und bleibt ausschließlich privat.*

Kapitel 8 – Was bleibt

Für die Weiterarbeit liegen 26 dokumentierte Restpunkte samt Priorisierung bereit ([Roadmap](#)); für das Verständnis dieses Buch und seine Schwesterdokumente; für die Sicherheit die Regeln, die sich bewährt haben: Sollwerte vor Code, Ehrlichkeit vor Vollständigkeit, Prozess vor Vertrauen, privat bleibt privat.

Anhang A – Architektur auf einer Seite

Storage → Provider (Reducer, injizierter Stichtag) → reine Finanzfunktionen → React-Seiten. Eine JSON-Datei, ein Hook (`useFinanceData`), ein Änderungspfad (`applyDataChange`), zwei Laufzeit-Dependencies. Details: [wiki/Architecture](#).

Anhang B – Zeitleiste

2026-07-19: Phasen 0–12 (Analyse → Entscheidungen → Doku → Fundament → M8/M9/M7).

2026-07-20: Phasen 13–19 (M10, M12, M11, M13, M14 → Feature Complete → Release-Tooling).

2026-07-21: Phase 20 (Finalabnahme, Tag v1.0.0) und dieses Archiv. Tabellarisch mit Testzahlen: [Development History §10](#).

Anhang C – Kennzahlen des Projekts

55 Quelldateien (~17.000 Zeilen), 37 Testdateien (815 Tests, ~12.600 Zeilen), 9 verbindliche Dokumente, 12 Commits, 1 lokaler Tag, 0 Netzwerkzugriffe, 2 Laufzeit-Abhängigkeiten, 26 offene V1.x-Punkte, 4 Nutzerentscheidungen, 12 globale Regeln, 22 Formeln.